CASE STUDY · CURRENT PROJECT STATUS

OS Tutor: Fine-Tuning a Small Language Model into a Verified Operating Systems Tutor

A 4-billion-parameter model that explains operating-system concepts, teaches Socratically, and delegates every calculation to deterministic simulators.

AUTHOR

Divyakush Punjabi

REGISTER NUMBER

24BCB0126

SUMMARY

General-purpose language models state the rules of operating systems correctly but fail to execute them over many steps, which makes them unreliable tutors for a subject assessed through worked numerical problems. This project fine-tunes Qwen3-4B-Instruct-2507 on an 11,760-row dataset built by an eleven-stage, gate-checked pipeline, and pairs the model with verified simulators that compute every procedural answer. The system is fully built and deployed locally. On 57 hand-written problems it scores 63.2% when the model answers alone and **100% (57/57)** with verified delegation. General knowledge is retained within 0.75 percentage points of the base model. One open weakness — declarative OS theory, 4.29 points below the base model — is the focus of the next phase.

28 September 2026

Contents

1. Introduction 3

1.1 Background 3

1.2 The problem 3

1.3 Project status at a glance 3

2. Literature Review 4

2.1 Parameter-efficient fine-tuning 4

2.2 Catastrophic forgetting 4

2.3 Limits of procedural reasoning in LLMs 4

2.4 Language models as tutors 5

2.5 Research gap 5

3. Objective 6

4. Dataset Collection 7

4.1 Collection strategy 7

4.2 Sources 7

4.3 Generated data 7

4.4 Licensing policy 8

4.5 Evaluation data, held apart 8

5. Dataset Processing 9

5.1 Pipeline overview 9

5.2 Analysis before cleaning 9

5.3 Cleaning, deduplication and decontamination 10

5.4 Quality scoring and gold selection 10

5.5 Final composition 11

5.6 Rectifier ordering and spaced replay 11

5.7 Release gates 12

6. Model Architecture Design 13

6.1 Design principle 13

6.2 Base model selection 13

6.3 Fine-tuning configuration 14

6.4 Verified-solver layer 14

6.5 Serving architecture 15

7. Conclusion 16

7.1 Current status: measured results 16

7.2 Findings 16

7.3 Limitations 17

7.4 Next steps 17

References 18

1. Introduction

1.1 Background

Operating systems is taught and examined largely through procedures. Students are asked to build a Gantt chart for Round Robin scheduling, count page faults under LRU, trace a disk head through SCAN, or decide whether a Banker's algorithm state is safe. Each is a fixed rule applied over many dependent steps, and a single wrong step invalidates every step after it.

Large language models are an attractive tutoring medium because they explain fluently, answer at any hour, and can adapt their explanation to a student's attempt. The question this project asks is whether a small model, trainable on free hardware and runnable on a student laptop, can be made *trustworthy* enough to tutor this subject.

1.2 The problem

A general model fails an OS student in three distinct ways:

- **It executes procedures unreliably.** It can state the rule for SRTF correctly and still produce the wrong schedule. In our own measurements, every one of the 7 scheduling failures on a hand-written suite contained **correct arithmetic paired with a wrong schedule**.
- **It gives answers away.** A student who shows a wrong attempt needs to be guided to the error, not handed the corrected answer.
- **Fine-tuning can damage it.** Training a model narrowly on OS material risks eroding the general ability that makes its explanations good — catastrophic forgetting.

The first failure is architectural rather than a gap in knowledge, as Section 2.3 shows. It cannot be solved by adding more training data: scheduling had the most training rows of any category (1,733) and the worst accuracy (42%), while disk scheduling had 990 rows and 100%.

1.3 Project status at a glance

Component	Status	Detail
Dataset pipeline	**Complete**	Version 3.0.0 — 11,760 training rows, all 19 release gates passed, reproducible build
Fine-tuned model	**Complete**	Qwen3-4B-Instruct-2507 + LoRA adapter v3.0, trained on Kaggle 2× T4
Verified-solver layer	**Complete**	Seven procedural families delegated to deterministic simulators
Serving stack	**Deployed**	Local llama.cpp + gateway + web interface, six tutoring modes, 26.9 tokens/s
Evaluation	**Complete**	Procedural suite, routing, catastrophic-forgetting check with paired flips
Declarative OS theory	**Open issue**	85.71% against 90.00% for the base model; remediation planned
Base-model upgrade study	**In progress**	2026-generation candidates identified; zero-shot probe pending (Section 7.4)

2. Literature Review

The design draws on four bodies of work: efficient fine-tuning, catastrophic forgetting, the limits of procedural reasoning in language models, and language models used for tutoring. Each subsection closes with how the finding shaped this project.

2.1 Parameter-efficient fine-tuning

LoRA [1] freezes the pretrained weights and learns a low-rank update, reducing trainable parameters by orders of magnitude. **QLoRA** [2] stores the frozen base in 4-bit NormalFloat and back-propagates through it into LoRA adapters, which makes fine-tuning feasible on a single consumer or datacentre GPU. **NEFTune** [3] adds uniform noise to embeddings during training and measurably improves instruction-following quality at no inference cost.

LIMA [4] showed that a small set of carefully curated examples can align a strong base model, supporting the view that data quality matters more than volume.

> **Applied here:** QLoRA made training possible on free Kaggle T4 GPUs; NEFTune ($\alpha = 5$) is enabled; and the LIMA principle informed a heavily curated pipeline — though applied to *knowledge* categories only, since procedural skill still benefits from many distinct worked examples.

2.2 Catastrophic forgetting

Neural networks trained sequentially on new data lose earlier competence, a phenomenon first characterised as catastrophic interference [5]. Regularisation approaches such as Elastic Weight Consolidation [6] penalise movement of weights important to earlier tasks, but require estimating parameter importance over the original training distribution, which is unavailable for a pretrained model.

Four recent results are more directly usable:

- **LoRA forgets less** than full fine-tuning, because the low-rank constraint bounds how far the weights can move [7].
- **Self-distillation (SDFT)** [8] rewrites training targets in the base model's own words, shrinking the distribution gap that drives forgetting.
- **SuRe** [9] prioritises replay of the most *surprising* sequences — those with the highest negative log-likelihood — which are the ones most at risk of being lost.
- **FOREVER** [10] observes that forgetting in language models follows an Ebbinghaus-like curve, and spaces replay frequently early in training and increasingly sparsely later.

> **Applied here:** the anti-forgetting design combines all four — LoRA as the structural constraint, SDFT targets for theory and tutoring, 16% replay data ranked by surprise, and replay spaced on a forgetting-curve schedule — and it is *measured* with paired item-level flips rather than assumed.

2.3 Limits of procedural reasoning in LLMs

Samiei et al. [11] measured long-horizon execution of explicit finite-state machines and found that even Qwen3-235B reaches only about **50%** task accuracy, degrading as the number of steps grows. The failure is in *executing* a known rule, not in knowing it.

Guo, Wu and Yiu [12] explain why. They bound the state-tracking capacity of decoder-only attention by $O(H \cdot \log(L/H) \cdot \sqrt{d_h})$, where H is the number of attention heads and d_h the head dimension, and identify a **deterministic horizon d* between 19 and 31 steps** beyond which extended chain-of-thought degrades. Across 12 models, tool-integrated reasoning reached 86–94% accuracy against 24–42% for purely neural chain-of-thought.

Kambhampati et al. [13] argue on independent grounds that language models cannot reliably verify their own plans, and propose **LLM-Modulo** systems in which the model generates and an external, sound verifier checks.

> **Applied here:** this is the central design decision of the project. Because the limit is architectural, no amount of fine-tuning removes it — so the model is never asked to compute. It reads the question, deterministic simulators compute the answer, and the model explains the result.

2.4 Language models as tutors

MathDial [14] collected tutoring dialogues in which teachers guide students who hold genuine misconceptions, and showed that off-the-shelf models tend to reveal answers too early rather than scaffolding the student's reasoning. Its annotated teacher moves — probing, focusing, telling — give a vocabulary for what good tutoring looks like.

> **Applied here:** MathDial supplies real teacher moves for the tutoring category, and the OS half of that category is built the same way: each dialogue starts from a wrong attempt generated from a *named* misconception, and any reply that reveals the final answer is rejected.

2.5 Research gap

Work	Finding	Gap this project addresses
LoRA, QLoRA [1, 2]	Fine-tuning fits on one GPU	Applied at 16,384-token sequences on free T4 hardware
SDFT, SuRe, FOREVER [8–10]	Individual anti-forgetting techniques	Combined in one pipeline and verified with paired flips
Illusion of Procedural Reasoning [11]	Even 235B models execute procedures poorly	Diagnosed in a real teaching domain, not a synthetic task
Deterministic Horizon, LLM-Modulo [12, 13]	External tools must carry deterministic work	Built as a working tutor with grounded parameter extraction
MathDial [14]	Models leak answers when tutoring	Prevented structurally at the routing layer, not by prompting

No prior work we found combines a fine-tuned small model, verified procedural delegation, enforced Socratic behaviour and measured forgetting resistance in a single deployable tutor for operating systems.

3. Objective

Primary objective: build an offline operating-systems tutor that is numerically exact, pedagogically honest, resistant to catastrophic forgetting, and trainable entirely on free hardware.

The primary objective breaks down into six measurable objectives. Each has a fixed target, and the current status is reported against it:

#	Objective	Target	Current status
O1	Exact answers to procedural problems	100% on a hand-written suite	**Achieved — 57/57**
O2	Socratic integrity: a shown attempt is taught, not answered	Correct routing of every attempt	**Achieved — 28/28, zero misroutes**
O3	No catastrophic forgetting of general ability	General MMLU within 1 pp of base	**Achieved — −0.75 pp**
O4	Declarative OS knowledge at least as good as base	OS theory MCQ ≥ base (90.00%)	**Not yet — 85.71%**
O5	Commercially clean, reproducible dataset	All release gates pass	**Achieved — 19/19**
O6	Free training and local interactive serving	Kaggle free tier; ≥ 20 tokens/s	**Achieved — 26.9 tokens/s**

> **What the tutor is not.** It is deliberately not a retrieval-augmented chatbot over a textbook. Retrieval returns text that *resembles* an answer; it cannot certify that a Gantt chart for a student's specific numbers is correct, because that chart exists in no document. The tutor computes the answer instead.

4. Dataset Collection

4.1 Collection strategy

Data was collected in five categories, each serving a distinct purpose in the finished tutor. The categories were defined before any data was gathered, so that collection followed the design rather than the reverse.

Category	Purpose
Procedural	Simulator-verified numerical problems, worked step by step, each decision written as a rule plus the comparison behind it
Conceptual	Explanations of OS concepts, fact-checked against a list of known misconceptions
Tutoring	Socratic dialogues that guide a student from a wrong attempt without revealing the answer
Shell	Natural language to bash commands — the practical side of an OS course
Replay	General instruction data, held as an anchor against catastrophic forgetting

4.2 Sources

In total **80,732 rows** were ingested from 31 source files, each row carrying line-level provenance (source file and row index) so that any training example can be traced to its origin.

Category	Sources	Rows ingested
Procedural	Project-built simulators (Section 4.3)	16,720
Conceptual	Hand-written 295-question bank in four wordings, self-distilled; hand-written gold explanations; misconception set	1,485
Tutoring	MathDial [14]; project-built OS dialogues from misconception-generated attempts	3,633
Shell	tldr pages, NL2Bash (MIT, TellinaTool), InterCode, NL2SH-ALFA	21,066
Replay	Tülu 3 SFT mixture [16] (commercial-safe subset), SmolTalk2 no-think	37,828
Total		**80,732**

4.3 Generated data

Public data for OS *numericals* is scarce and rarely verified, so the procedural category is generated by simulators written for this project. Every answer is computed, never written by hand or by a model. The simulators cover **39 question types across 8 families**:

- **CPU scheduling** — FCFS, SJF, SRTF, Round Robin, priority, preemptive priority, HRRN
- **Page replacement** — FIFO, LRU, OPT, CLOCK
- **Deadlock** — Banker's safety and request, including unsafe states
- **Disk scheduling** — FCFS, SSTF, SCAN, LOOK, C-SCAN, C-LOOK
- **Contiguous allocation** — first, best and worst fit
- **Address translation**, and **11 formula kinds** — effective access time, multilevel paging, deadlock minimum, fork count and others

- **Clarification** — problems deliberately missing a required input, where the correct behaviour is to ask rather than guess

Two further categories are partly generated. Conceptual answers are **self-distilled** [8] from the base model, then sanitised and fact-checked; any answer asserting a known misconception is dropped, never repaired. OS tutoring dialogues begin from a wrong attempt produced by a *named* misconception — for example, applying non-preemptive SJF to an SRTF question.

4.4 Licensing policy

Every training row carries a licence that permits commercial use. The shipped split contains MIT 8,991 rows, ODC-BY-1.0 959, mixed-permissive 925 and CC-BY-4.0 885. Three sources were excluded, and the reason is recorded for each:

Excluded source	Reason
ostep-homework	No declared licence — used only as an offline test oracle, never ingested
ceval/ceval-exam	CC BY-NC-SA — evaluation only, never training
AnishJoshi/nl2bash-custom	No declared licence — superseded by the MIT-licensed TellinaTool data

4.5 Evaluation data, held apart

Evaluation sets were collected separately and never trained on. They are ingested only so that the pipeline can remove any training row that overlaps them (Section 5.3).

Evaluation set	Items	Measures
Hand-written procedural suite	57	Exact final answers across 19 question types
Hand-written OS theory MCQ	70	Declarative OS knowledge
MMLU, general subjects	400	Retention of general knowledge (forgetting)
MMLU, adjacent computer science	300	Transfer to neighbouring subjects
Out-of-distribution holdout	600	Generalisation to unseen problem framings

5. Dataset Processing

5.1 Pipeline overview

Raw data passes through eleven stages, run by a single command. Each stage writes its statistics to a manifest, so every number in this section is taken from the build record rather than estimated.

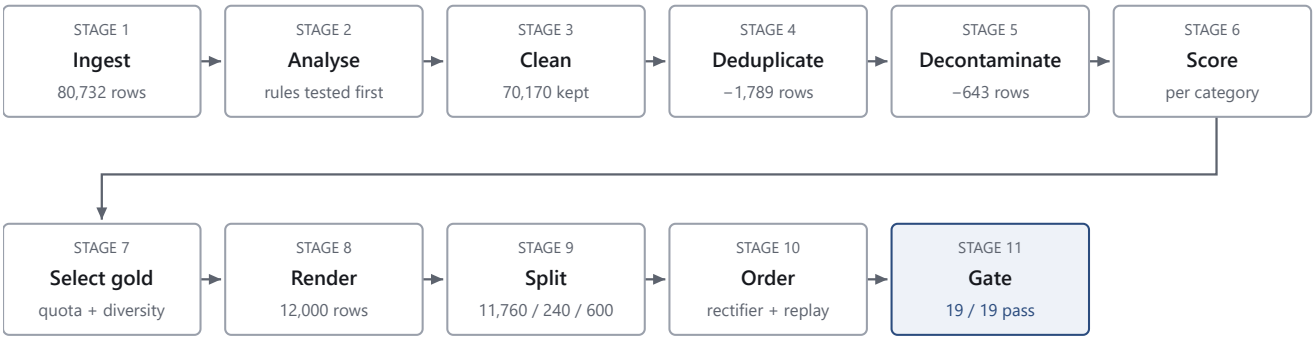


Figure 1 — The eleven-stage dataset pipeline. A build is released only if every gate in stage 11 passes.

5.2 Analysis before cleaning

Before any row is removed, each category is analysed on its own. Every candidate cleaning rule is tested against the data and adopted only if the evidence supports it — so the rules follow the data rather than an assumption about it. The analysis found, for example:

Category	Defects found	Rules rejected on evidence
Procedural	434 missing a final-answer marker; 278 with no visible steps	—
Tutoring	283 non-alternating roles; 262 dead tails; 212 never ask the student a question; 31 leak the answer in the first turn	“answer shorter than question”
Shell	181 vague instructions; 126 bash syntax errors; 70 editor keystrokes	“ends mid-sentence”, “brace placeholder”

Rejecting a rule is as important as adopting one. A tutoring rule that dropped replies shorter than the question looked sensible, but the analysis showed it would remove good Socratic prompts, which are short by design.

5.3 Cleaning, deduplication and decontamination

Cleaning kept 70,170 of 80,732 rows. The largest reasons for removal were:

Reason	Rows removed
Replay answer too short to teach anything	3,617
Assistant turn too short	1,515
Replay in a non-Latin script	1,494
Degenerate repetition	1,320
Refusals	1,214
Tutoring reply that never asks the student a question	212
Shell command that fails `bash -n`	126
Credential or secret material (destroyed, not redacted)	54

A further 261 rows were repaired rather than dropped, and 707 pieces of personal information were redacted — 377 card numbers (Luhn-checked), 180 e-mail addresses, 149 phone numbers and one social-security number.

Deduplication removed 656 exact duplicates (blake2b hashing) and 1,133 near-duplicates (MinHash LSH at Jaccard 0.80).

Decontamination compared training data against all 4,836 evaluation items and removed 643 rows. Prose rows are matched on distinctive 13-grams, ignoring 93 boilerplate n-grams common to many rows. Generated problems are matched on an exact *parameter fingerprint*, which also catches a reworded leak of the same problem — 15 such matches were found. Without this step, evaluation scores would be inflated by memorised answers.

5.4 Quality scoring and gold selection

Every surviving row is scored by a scorer specific to its category. Mean quality after selection:

Category	Mean quality	Pairwise diversity
Procedural	0.833	0.599
Tutoring	0.804	0.694
Replay	0.767	0.958
Shell	0.728	0.944
Conceptual	0.692	0.708

Gold selection then applies a quality floor, removes semantic duplicates (328 from the procedural pool), and uses farthest-point diversity selection so that the chosen rows cover the space rather than cluster. Procedural selection is a weighted allocation over question families, so a scarce family cannot be crowded out by a common one.

5.5 Final composition

Category	Designed share	Rows	Realised share	Drift
Procedural	64%	7,526	64.0%	−0.00 pp
Replay	16%	1,884	16.0%	+0.02 pp
Tutoring	8%	938	8.0%	−0.02 pp
Shell	6%	707	6.0%	+0.01 pp
Conceptual	6%	705	6.0%	−0.01 pp
Training split	**100%**	**11,760**		

A further 240 rows form the validation split and 600 the out-of-distribution holdout. Within the procedural category, rows are distributed across families as follows: CPU scheduling 1,733; formula 1,530; deadlock and Banker's 1,070; page replacement 990; disk scheduling 990; contiguous allocation 536; address translation 382; clarification 295.

Rendered in the model's own tokenizer, rows average 689 tokens (median 603, 99th percentile 2,753, longest 11,648). Training uses 16,384-token sequences, above the longest row, so **no training example is ever truncated** and no worked answer loses its final line.

5.6 Rectifier ordering and spaced replay

Training data is read in a fixed order rather than shuffled, and that order is itself engineered. The **rectifier** interleaves categories so that no skill is learned in one uninterrupted block and then decays while another is learned:

Ordering constraint	Limit	Measured
Longest run of one category	4	4
Longest run from one source dataset (81 sources)	3	1
Replay spacing, early → late in training	—	every 4 → every 14 rows
Spacing violations	0	0

Replay is spaced densely early and sparsely later, following the forgetting-curve schedule of FOREVER [10], and within the replay pool examples are ranked by surprise, following SuRe [9].

5.7 Release gates

A build is released only if all 19 automated gates pass, checked on the rows actually shipped rather than on the plan. A failing gate stops the build; it does not emit a warning. The current build passes all 19.

Group	Gates	Result
Safety and licence	Zero secrets · zero non-commercial rows · no hidden reasoning blocks · no non-Latin rows	4 / 4
Integrity	Schema valid · zero known defects · 99th-percentile length within limit · no procedural question missing an input	4 / 4
Mixture	Drift under 2 pp · minimum rows per category · minimum total rows · family weights honoured · OS tutoring present	5 / 5
Diversity	Distinct-answer ratio · template share · pairwise diversity	3 / 3
Ordering	Rectifier invariants · bounded single-source runs	2 / 2
Evaluation	Genuine out-of-distribution holdout present	1 / 1
Total		**19 / 19**

The build is reproducible: the training file is content-addressed by its SHA-256 hash (`f56a7113e4aa5c0e…`), and the same configuration and seed reproduce it exactly.

6. Model Architecture Design

6.1 Design principle

The model explains; simulators compute. Following Section 2.3, the language model is never asked to carry out a procedure it cannot execute reliably. A fine-tuned 4B model handles reading, language and pedagogy; deterministic simulators handle every number. They are joined by an extraction step that turns a question into typed parameters.

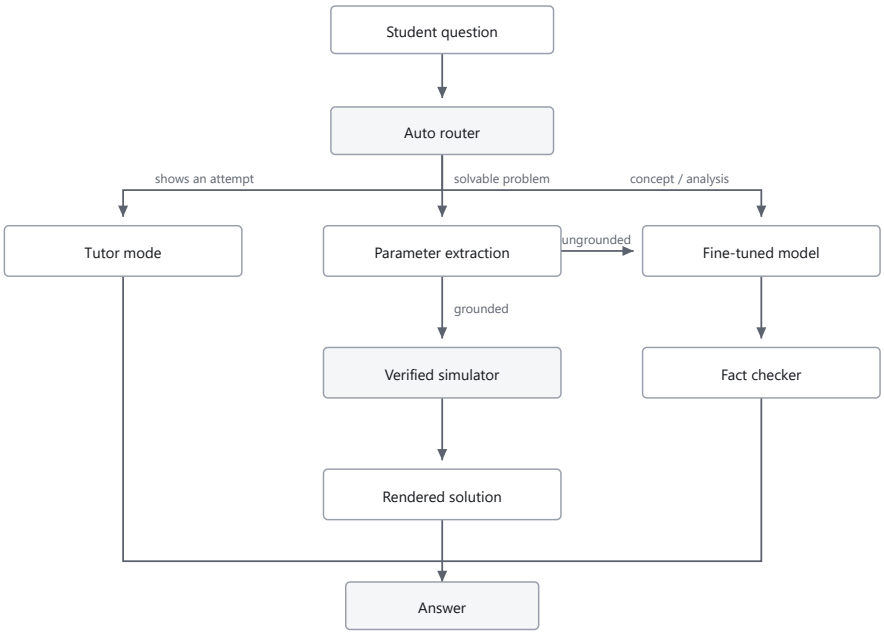


Figure 2 — Request flow. Shaded nodes are deterministic; the model is never asked to compute.

The router reads the question first. A shown attempt outranks every other signal and routes to tutoring — and the tutoring path has no access to the solver or the fact checker, so there is no route by which a computed answer can reach a student who is still working.

6.2 Base model selection

The base model is **Qwen3-4B-Instruct-2507** [15], Apache-2.0 licensed. It was selected for four reasons: its non-thinking variant writes reasoning as visible steps a student can read; its 262,144-token native context comfortably covers 16,384-token training sequences; at 4B it fits QLoRA training on a 16 GB T4; and it scores strongly for its size — MMLU-Pro 69.6, MMLU-Redux 84.2 and GPQA 62.0 on its model card, where it is benchmarked against the 30B-parameter Qwen3-30B-A3B.

A study of the 10–20B range confirmed the choice for the current hardware. Figures below are estimates or vendor claims, not measurements on our machines:

Alternative	Reason not selected
Dense 14B (Qwen3-14B, Phi-4)	About 16–17 GB for QLoRA at 16,384 tokens — over a single T4 — and about 5 tokens/s served from 8 GB VRAM
gpt-oss-20b (MoE)	About 23 GB for training at 16,384 tokens; reasoning is hidden in a separate channel
Qwen3.5-9B	22 GB for 16-bit LoRA; its vendor advises against 4-bit QLoRA training

6.3 Fine-tuning configuration

Setting	Value	Reason
Method	QLoRA via Unsloth	4-bit base keeps a 16k-token sequence inside 16 GB
LoRA rank / alpha	32 / 64, all linear layers	Capacity for eight procedural families without full-weight drift
Dropout, learning rate	0.05, 1×10^{-4}, cosine	Conservative update size for a small base
Epochs	1	A second epoch memorised more and generalised less in version 2.2
Sequence length	16,384 tokens, no packing	No row truncated; packing would break the rectifier order
Precision, optimiser	fp16, 8-bit AdamW	T4 (Turing) has no bf16 support
Regularisation	NEFTune $\alpha = 5$ [3]	Better instruction quality at no inference cost
Loss masking	Assistant tokens only	The model learns to answer, not to echo the prompt
Sampler	Sequential	Shuffling would destroy the engineered order
Hardware	Kaggle, 2 × T4, DDP	Free tier; no paid compute anywhere

Training behaved as intended. Validation loss fell monotonically from 0.4413 to 0.2713 over the epoch, and the final training loss (0.2710) sits almost exactly on the validation loss (0.2754) — the signature of a model that fitted the task without memorising its training traces.

6.4 Verified-solver layer

The solver turns a 63% model into a 100% system in three stages:

1. **Extract.** The question is parsed into typed JSON parameters. Extraction deliberately runs on the *untuned* base model, which read parameters correctly 84% of the time against 79% for the fine-tuned adapter — the adapter is biased toward solving rather than reading.
2. **Ground.** Every extracted number must appear in the question. An invented parameter causes the question to fall back to the model; a missing input produces a clarifying question instead of a guess.
3. **Solve and render.** A deterministic simulator computes the result, and the answer is written in the same worked-steps format the model was trained on, so the student sees no seam.

A separate **fact checker** holds ten canonical corrections for misconceptions observed in real model answers — for instance, that LRU never suffers from Belady's anomaly — and rewrites a draft that asserts one.

6.5 Serving architecture

Layer	Component
Weights	llama.cpp (CUDA), Q8_0 GGUF base with the LoRA adapter applied at runtime
Logic	FastAPI gateway — routing, solver delegation, fact checking, repetition-loop guard
Interface	Open WebUI, running locally with no external calls

Six modes are offered to the student:

Mode	Behaviour
`ostutor` (default)	Auto mode — reads the question and routes it to the right mode below
`solve`	Full worked solution, computed by the simulators
`analyze`	Open comparison and reasoning, e.g. "why is Round Robin better for interactive work?"
`explain`	Concept explanation, fact-checked
`tutor`	Socratic guidance on the student's own attempt, never revealing the answer
`shell`	Natural language to bash commands

On the development laptop (RTX 4060, 8 GB) the model generates at a measured **26.9 tokens per second**, faster than a student reads.

7. Conclusion

The project has reached a working, deployed system. Its central claim — that a small fine-tuned model can tutor operating systems exactly if it delegates computation — is now supported by measurement. One weakness is open, and it is reported here in full.

7.1 Current status: measured results

Procedural accuracy, on 57 hand-written problems graded by simulator-exact comparison:

Question type	Model alone	With verified solver
CPU scheduling	5 / 12	**12 / 12**
Page replacement	5 / 8	**8 / 8**
Disk scheduling	4 / 7	**7 / 7**
Banker's safety	3 / 4	**4 / 4**
Free-space management	0 / 4	**4 / 4**
Address translation	1 / 3	**3 / 3**
Clarification required	3 / 3	**3 / 3**
Remaining types (12 kinds)	15 / 16	**16 / 16**
Total	**36 / 57 (63.2%)**	**57 / 57 (100%)**

Catastrophic forgetting, on 770 identical items scored before and after fine-tuning. Flips are reported because an unchanged average can hide many cancelling changes:

Benchmark	Base	Fine-tuned	Change	Right → wrong	Wrong → right
General MMLU (400)	80.25%	79.50%	−0.75 pp	13	10
Adjacent CS (300)	64.67%	67.33%	+2.66 pp	11	19
OS theory (70)	90.00%	85.71%	−4.29 pp	4	1

System verification: routing 28 / 28 on labelled cases; data-pipeline tests 89 checks passed; serving-gateway tests 25 checks passed; dataset release gates 19 / 19.

7.2 Findings

1. **The accuracy ceiling was execution, not knowledge.** Delegating computation lifted the suite from 63.2% to 100% with no retraining, and cut a full evaluation from 76 minutes to about 150 seconds.
2. **The anti-forgetting design worked on general ability.** General knowledge moved 0.75 points, within scoring noise, and adjacent computer science *improved*, with 19 items gained against 11 lost.
3. **Socratic behaviour is enforceable by architecture.** Routing, not prompting, guarantees that a student's attempt is never met with a computed answer.
4. **Procedural skill rose while declarative OS recall dipped.** This was not anticipated: the loss that was feared in general knowledge appeared instead, slightly, in OS theory.

7.3 Limitations

- The OS theory result rests on 70 questions, where four items decide the difference, and the base model was already near ceiling at 90%. It needs a larger set before it can be called settled.
- Comparisons of the base model and the previous adapter on the procedural suite are outstanding. An earlier measurement was invalidated by a harness fault and withdrawn rather than reported.
- The conceptual half of tutoring has no external verifier and no rubric-based measurement yet.
- The fact checker holds a finite list of corrections and cannot catch an error nobody has written a rule for.

7.4 Next steps

Next step	Purpose
Zero-shot probe of 2026 base models on OS theory — Qwen3.5-4B and Gemma 4 12B, both Apache-2.0	Decide whether a newer base improves declarative knowledge before any retraining (about 3 GPU-hours)
Expand the OS theory set from 70 to 200 questions	A stable estimate that does not turn on four items
Targeted declarative replay for the regressed topics	Recover OS theory accuracy to at least the base model's
Extend delegation to the 11 formula kinds	Remove the remaining model-answered procedural items
Rubric-scored evaluation of conceptual tutoring	Measure the one area with no number today

> **In summary,** the project set out to test whether a model small enough for a student laptop could tutor operating systems exactly. It can, provided the model is never trusted with arithmetic it cannot perform. The system is built, deployed and measured; the remaining work is to raise declarative OS knowledge to match the procedural accuracy already achieved.

References

1. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. “LoRA: Low-Rank Adaptation of Large Language Models.” *ICLR*, 2022. arXiv:2106.09685.
2. T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. “QLoRA: Efficient Finetuning of Quantized LLMs.” *NeurIPS*, 2023. arXiv:2305.14314.
3. N. Jain et al. “NEFTune: Noisy Embeddings Improve Instruction Finetuning.” *ICLR*, 2024. arXiv:2310.05914.
4. C. Zhou et al. “LIMA: Less Is More for Alignment.” *NeurIPS*, 2023. arXiv:2305.11206.
5. M. McCloskey, N. J. Cohen. “Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem.” *Psychology of Learning and Motivation*, vol. 24, 1989.
6. J. Kirkpatrick et al. “Overcoming Catastrophic Forgetting in Neural Networks.” *Proceedings of the National Academy of Sciences*, 114(13), 2017.
7. D. Biderman et al. “LoRA Learns Less and Forgets Less.” *Transactions on Machine Learning Research*, 2024. arXiv:2405.09673.
8. Z. Yang et al. “Self-Distillation Bridges Distribution Gap in Language Model Fine-Tuning.” *ACL*, 2024. arXiv:2402.13669.
9. H. Hazard, Z. Fountas et al. “SuRe: Surprise-Driven Prioritised Replay for Continual LLM Learning.” arXiv:2511.22367, 2025.
10. “FOREVER: Forgetting Curve-Inspired Memory Replay for Language Model Continual Learning.” arXiv:2601.03938, 2026.
11. M. Samiei, M. Mansouri, M. Soleymani Baghshah. “The Illusion of Procedural Reasoning: Measuring Long-Horizon FSM Execution in LLMs.” arXiv:2511.14777, 2025.
12. D. Guo, J. Wu, S. M. Yiu. “The Deterministic Horizon: When Extended Reasoning Fails and Tool Delegation Becomes Necessary.” arXiv:2606.00376, 2026.
13. S. Kambhampati et al. “Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks.” *ICML*, 2024. arXiv:2402.01817.
14. J. Macina et al. “MathDial: A Dialogue Tutoring Dataset with Rich Pedagogical Properties Grounded in Math Reasoning Problems.” *Findings of EMNLP*, 2023. arXiv:2305.14536.
15. Qwen Team. “Qwen3 Technical Report.” arXiv:2505.09388, 2025.
16. N. Lambert et al. “Tülu 3: Pushing Frontiers in Open Language Model Post-Training.” arXiv:2411.15124, 2024.